

■ shopEZ #### A Full-Stack MERN E-Commerce Marketplace *Sleek. Scalable. Secure.* ---
[Node.js](https://img.shields.io/badge/Node.js-18%2B-339933?style=for-the-badge&logo=node.js&logoColor=white))(https://nodejs.org)
[React](https://img.shields.io/badge/React-19-61DAFB?style=for-the-badge&logo=react&logoColor=black))(https://react.dev)
[MongoDB](https://img.shields.io/badge/MongoDB-8.x-47A248?style=for-the-badge&logo=mongodb&logoColor=white))(https://mongodb.com)
[Express](https://img.shields.io/badge/Express-4.x-000000?style=for-the-badge&logo=express&logoColor=white))(https://expressjs.com)
[Tailwind CSS](https://img.shields.io/badge/Tailwind-3.x-06B6D4?style=for-the-badge&logo=tailwindcss&logoColor=white))(https://tailwindcss.com)
[Vite](https://img.shields.io/badge/Vite-8.x-646CFF?style=for-the-badge&logo=vite&logoColor=white))(https://vitejs.dev)

■ Table of Contents

- [Overview](#)
- [Tech Stack](#)
- [Feature Highlights](#)
- [Project Structure](#)
- [Quick Start](#)
- [Environment Variables](#)
- [API Reference](#)
- [Phase-Wise Documentation](#)
- [Future Enhancements](#)

■ Overview

shopEZ is a production-grade, fully decoupled **MERN stack e-commerce marketplace** that delivers a seamless online shopping experience for customers alongside a powerful administrative control center for store managers.

Built with a strict **Model-View-Controller (MVC)** architecture, shopEZ showcases best practices in modern full-stack development: - **Stateless JWT authentication** with RBAC (Customer / Seller / Admin) - **Reactive SPA frontend** built with React 19, Vite, and Tailwind CSS - **RESTful Express API** with centralized error handling and input validation - **MongoDB + Mongoose** for flexible, schema-validated document storage - **Mock Stripe Payment Intent** endpoint for a production-grade checkout flow

■ Tech Stack

Layer	Technology	Purpose
-------	------------	---------

Frontend	React 19 + Vite	SPA with virtual DOM, HMR dev server
Styling	Tailwind CSS 3	Utility-first responsive design
Routing	React Router DOM 6	Declarative routes with protected guards
HTTP Client	Axios	REST API communication with auth interceptors
Charts	Chart.js + react-chartjs-2	Admin sales analytics visualizations
Backend	Node.js 18 + Express.js	RESTful API, middleware, business logic
Database	MongoDB + Mongoose	NoSQL document storage with ODM
Auth	JWT + Bcrypt.js	Stateless tokens + secure password hashing
File Uploads	Multer	Local image handling for products
Dev Fallback	mongodb-memory-server	In-memory DB when no MongoDB connection exists

■ Feature Highlights

■ Authentication & Access Control

- Secure **JWT session tokens** stored in HTTP-only cookies
- **Role-Based Access Control** with three tiers: `customer`, `seller`, `admin`
- Protected API routes via Express middleware; Protected frontend routes via `ProtectedRoute.jsx`
- **Profile Dashboard**: Edit personal info, manage multiple saved shipping addresses

■ Product Catalog & Discovery

- **Live Autocomplete** suggestions on search input
- Dynamic filters: **Category**, **Price Range**, **Minimum Rating**, **In-Stock Only**
- Multiple **sort modes**: Price (low/high), Rating, Newest first

- **Product Detail View:** Image gallery, variant selection, related products carousel, delivery estimator

■ Cart & Wishlist

- **Persistent shopping cart** preserved across page reloads via React Context + localStorage
- Real-time **stock validation** on quantity changes
- **Wishlist** to save favorites with one-click **"Move to Cart"** that auto-clears the wishlist entry

■ Checkout & Payments

- **Multi-step checkout wizard:**
 - Choose saved shipping address
 - Select shipping speed (Standard / Express)
 - Apply promotional coupon code
 - Review itemized totals (subtotal + discount + tax + shipping)
 - Mock Stripe credit card entry & order placement
- **Coupon Engine:** Validates active codes, applies **fixed** or **percentage** discounts
- **Mock Stripe Payment Intent** at </api/payment/create-intent> — simulates production-grade Stripe handshake

■ Order Management & Tracking

- Full **order history** with itemized invoices
- **6-step visual timeline tracker:** Placed -> Confirmed -> Packed -> Shipped -> Out for Delivery -> Delivered

■ Reviews & Ratings

- **Verified Purchase** badge automatically applied when reviewing an ordered product
- **Star ratings** recalculate the product's aggregate score on each new submission
- **Helpful Upvote** system to surface the most useful community reviews

■■ Admin & Seller Dashboards

- **Live Analytics:** Monthly revenue charts, category share doughnuts, active user growth lines, low-stock alerts (Chart.js)
- **Product CRUD:** Create, edit, delete listings; upload images via Multer
- **Orders Panel:** Override fulfillment milestones for any order

- **Customer Panel:** Instantly block/unblock user accounts
- **Coupon Manager:** Create, toggle, and delete promotional discount rules

■ Project Structure

```
E-Commerce Application/
|
|-- backend/                # Node.js + Express API
|   |-- config/db.js        # MongoDB connection + in-memory fallback
|   |-- controllers/        # Business logic (auth, products, orders, coupons)
|   |-- middleware/         # JWT auth guards, error handler
|   |-- models/             # Mongoose schemas (User, Product, Order, Review, Coupon, Seller)
|   |-- routes/             # Express route definitions
|   |-- utils/seeder.js     # Database seed script
|   |-- uploads/            # Local product image storage
|   |-- server.js           # Express app entry point
|   |-- .env.example        # Environment variable template
|
|-- frontend/               # React 19 + Vite SPA
|   |-- src/
|       |-- components/     # Navbar, ProductCard, ProtectedRoute
|       |-- context/        # AuthContext, CartContext, ToastContext
|       |-- pages/          # Home, ProductDetails, Cart, Wishlist, Checkout,
|       |                   # MyOrders, ProfileDashboard, AdminDashboard,
|       |                   # SellerDashboard, Login, Register
|       |-- utils/          # Axios config, helpers
|       |-- App.jsx         # Root route tree
|       |-- main.jsx        # ReactDOM bootstrap
|
|-- document/               # ■ Generated PDF Project Documentation
|   |-- 1_Brainstorming_and_Ideation.pdf
|   |-- 2_Requirement_Analysis.pdf
|   |-- 3_Project_Design.pdf
|   |-- 4_Project_Planning.pdf
|   |-- 5_UAT_Testing.pdf
|   |-- 6_FSD_Project_Documentation.pdf
|   |-- Frontend_README.pdf
|   |-- Project_README.pdf
|   |-- Project_Walkthrough.pdf
```

■ Quick Start

Prerequisites

- **Node.js** v18+ and **npm** v8+

- **MongoDB** (local or Atlas)

1 — Clone & Enter

```
git clone https://github.com/<your-username>/E-Commerce-Application.git
cd "E-Commerce Application"
```

2 — Configure Backend Environment

```
cd backend
copy .env.example .env      # Windows
# cp .env.example .env      # Mac/Linux
```

Edit `.env` with your values (see [Environment Variables](#)).

3 — Install Dependencies

```
# Backend
cd backend && npm install
|
# Frontend
cd ../frontend && npm install
```

4 — (Optional) Seed the Database

```
cd backend
npm run seed
```

Seeds 20+ products, admin account, customers, sample reviews, and active coupon codes.

5 — Run the App

```
# Terminal 1 — API Server (port 5000)
cd backend && npm run dev
|
# Terminal 2 — Frontend (port 5173)
cd frontend && npm run dev
```

Open **`http://localhost:5173`** in your browser. ■

Default Seeded Accounts

Role	Email	Password
Admin	admin@shopez.com	admin123
Customer	customer@shopez.com	customer123

■ Environment Variables

Create `/backend/.env` with the following keys:

```
# Server
PORT=5000

# Database
MONGO_URI=mongodb://127.0.0.1:27017/shopez

# Authentication
JWT_SECRET=your_super_secret_jwt_key_here

# Payments (Stripe – mock mode by default)
STRIPE_API_KEY=your_stripe_api_key_here
```

Tip: If `MONGO_URI` is unreachable, the app automatically starts an `mongodb-memory-server` in-memory fallback and seeds it with demo data — no setup needed for local testing!

■ API Reference

All routes are served from `http://localhost:5000`. Protected routes require `Authorization: Bearer <token>`.

Prefix	Description
<code>/api/users</code>	Registration, login, profile, wishlist, addresses, admin user management
<code>/api/products</code>	Catalog listing, CRUD, autocomplete suggestions, reviews, helpful votes
<code>/api/orders</code>	Place orders, view history, update payment/delivery status

/api/coupons	Admin coupon CRUD + user coupon validation
/api/upload	Multer image upload endpoint
/api/analytics	Sales, revenue, user growth metrics for Admin Dashboard
/api/payment/create-intent	Mock Stripe payment intent generation

■ See the full API table with methods and auth requirements in [document/6_FSD_Project_Documentation.pdf](#)

■ Project Documentation (PDFs)

All the project documentation has been compiled into professional PDF manuals and stored in the [document/](#) folder:

Category	Document	Description
Phase 1	Brainstorming & Ideation (PDF)	Problem statements, idea listing, MoSCoW priority matrix
Phase 2	Requirement Analysis (PDF)	User personas, empathy maps, functional & non-functional requirements
Phase 3	Project Design (PDF)	Problem-solution fit, MVC architecture, DFD Level 0 & Level 1
Phase 4	Project Planning (PDF)	Agile product backlog, user stories, sprint schedules, velocity
Phase 5	UAT Testing (PDF)	Full user acceptance test cases with expected results
Phase 6	FSD Project Documentation (PDF)	Setup guide, folder structure, full API reference, security model
Walkthrough	Project Walkthrough (PDF)	Detailed walkthrough of the implemented feature flows

Readme	Project README (PDF)	PDF version of the main README guide
Frontend	Frontend README (PDF)	Subdirectory README explaining React architecture

■ Future Enhancements

- [] **Real Stripe Integration** — Live payment processing with webhook confirmation
- [] **Cloudinary CDN** — Replace Multer disk storage with cloud-based image hosting
- [] **AI Recommendations** — Collaborative filtering for personalized product suggestions
- [] **Multi-Vendor Marketplace** — Independent seller profiles with commission tracking
- [] **Email Notifications** — Order status emails via Nodemailer or SendGrid
- [] **Push Notifications** — Browser push on shipment and delivery events
- [] **Advanced Analytics** — Cohort retention charts and inventory forecasting

****shopEZ**** — Built with ❤️ using the MERN Stack